

# RUNBOOK — How to Run Everything and Reproduce Every Figure

**HACKSIMUL8 2026 · 6-DOF UAV Quadcopter** For anyone who has to demonstrate this work, answer "*how did you get that graph?*", or re-run it from scratch.

## 0. Before you start

```
cd 'C:\Users\LENOVO\Downloads\HACKSIMU8\solution'
```

**Every command below assumes you are in that folder.** All scripts read their parameters from `quad_params.m` — nothing is hard-coded anywhere else, so changing a parameter there propagates through the whole project.

### Required toolboxes

| Toolbox                    | Used for                                                                                             |
|----------------------------|------------------------------------------------------------------------------------------------------|
| MATLAB                     | everything                                                                                           |
| Simulink                   | <code>quad_altitude.slx</code>                                                                       |
| Control System Toolbox     | <code>tf</code> , <code>ss</code> , <code>piddtune</code> , <code>margin</code> , <code>pzmap</code> |
| Deep Learning Toolbox      | <code>fitnet</code> , <code>train</code> (Task 3)                                                    |
| Parallel Computing Toolbox | <code>parfor</code> in Tasks 3 and 4 (optional — falls back to serial)                               |
| Simulink Control Design    | PID Tuner, <code>linearize</code>                                                                    |
| Stateflow                  | installed, not currently used                                                                        |

Check with `smoke_test` — it reports each one.

## 1. The 30-second demo (if a judge says "show me")

```
verify_all          % 25 checks, ~2 min, all should pass
open_system('quad_altitude') % then press Run, double-click the Scope
```

`verify_all` prints a live pass/fail for every claim in the documentation. It **re-derives** results rather than loading saved ones, so it is genuine evidence, not a replay.

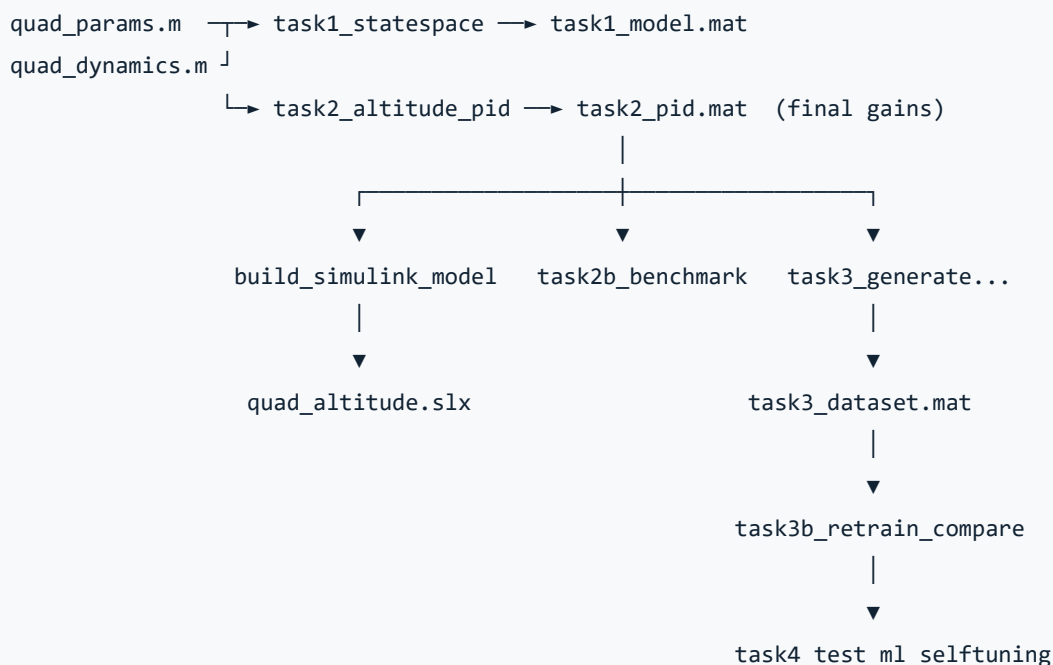
## 2. Full reproduction, in dependency order

Run these in sequence. Later scripts consume earlier scripts' `.mat` files.

| #  | Command                                   | Time    | Produces                                                      |
|----|-------------------------------------------|---------|---------------------------------------------------------------|
| 1  | <code>smoke_test</code>                   | ~30 s   | 10-point sanity check                                         |
| 2  | <code>task1_statespace</code>             | ~5 s    | <code>task1_model.mat</code>                                  |
| 3  | <code>task2_altitude_pid</code>           | ~60 s   | <code>task2_pid.mat</code> ← <b>the final gains</b>           |
| 4  | <code>task2b_zn_tyreus_luyben</code>      | ~30 s   | <code>task2b_benchmark.mat</code>                             |
| 5  | <code>build_simulink_model</code>         | ~10 s   | <code>quad_altitude.slx</code>                                |
| 6  | <code>task3_generate_data_train_ml</code> | ~13 min | <code>task3_dataset.mat</code> , <code>task3_model.mat</code> |
| 7  | <code>task3b_retrain_compare</code>       | ~60 s   | overwrites <code>task3_model.mat</code> with the best class   |
| 8  | <code>task4_test_ml_selftuning</code>     | ~5 min  | <code>task4_results.mat</code>                                |
| 9  | <code>export_figures</code>               | ~60 s   | all PNGs in <code>figures/</code>                             |
| 10 | <code>verify_all</code>                   | ~2 min  | 25-check verification                                         |

Or simply `run_all` for steps 2, 3, 6, 8.

### Dependency chain — why order matters



If you change `alt_cost.m` or the gains in `task2_pid.mat`, everything downstream of it is stale and must be re-run. This bit us once during development.

### 3. Every figure — which script made it, and what it shows

---

All figures live in `solution/figures/` at 150 dpi with a white background. `export_figures.m` regenerates all of them in one call.

#### `01_task1_openloop_divergence.png`

- **Made by:** `export_figures.m` (also plotted inline by `task1_statespace.m`)
- **Shows:** altitude with a +2% thrust step at  $t = 1$  s, no controller
- **How:** integrates `quad_dynamics` with `ode45` for 3 s at  $U1 = W \cdot (1 + 0.02 \cdot (t > 1))$
- **Point:** a 2% thrust error diverges as  $t^2$ . The quadcopter cannot fly open loop.

#### `02_task1_four_channels_pzmap.png`

- **Made by:** `export_figures.m`
- **Shows:** pole-zero maps of altitude, roll, pitch and yaw
- **How:** builds each channel transfer function from `quad_params` and calls `pzmap`
- **Point:** every channel is a double integrator — both poles at the origin.

#### `03_task2_tuning_comparison.png`

- **Made by:** `export_figures.m` (reads `task2_pid.mat`)
- **Shows:** step response and thrust for all four tuning methods
- **How:** replays each method's gains through `sim_altitude` for 8 s, overlays them
- **Point:** Method D (ITAE-optimised) is fastest with zero overshoot. The lower panel shows thrust touching the 20.248 N saturation limit — realistic, not a bug.

#### `04_feedback_linearisation_tilt.png`

- **Made by:** `export_figures.m`
- **Shows:** altitude at 0, 0.15, 0.30 and 0.45 rad of pitch; lower panel is the same data in millimetres
- **How:** `sim_altitude` with `opt.tilt_theta = @(t) tilt*(t > 3)` for each angle
- **Point:** altitude is essentially unaffected by tilt — sag stays in the micrometre range up to 25.8°.

#### `05_fixed_gains_degrade.png` ← **the bridge to Task 3**

- **Made by:** `export_figures.m`
- **Shows:** the same Task 2 gains at payload  $\times 1.0$ ,  $\times 1.4$ ,  $\times 1.8$ ,  $\times 2.2$
- **How:** `sim_altitude` with `P.m` scaled but `opt.m_ctrl` held at the nominal mass, so the controller does not know about the payload
- **Point:** up to 36 cm of steady-state error. This is why Task 3 exists.
- **Key detail if asked:** the `m_ctrl` option is what makes the payload genuinely unknown. Without it the controller compensates perfectly and the plot is flat — that was a real bug we found and fixed.

#### 06\_task3\_gains\_vs\_mass.png

- **Made by:** `export_figures.m` (reads `task3_dataset.mat`)
- **Shows:** optimal  $K_p$ ,  $K_i$ ,  $K_d$  against payload for all 150 sampled conditions
- **How:** scatter of the optimisation labels `Y` against the true mass ratio `COND(:,1)`
- **Point:**  $K_i$  rises clearly with payload;  $K_p$  and  $K_d$  scatter. That visual difference is the  $R^2$  story —  $K_i$  is identifiable, the others are not.

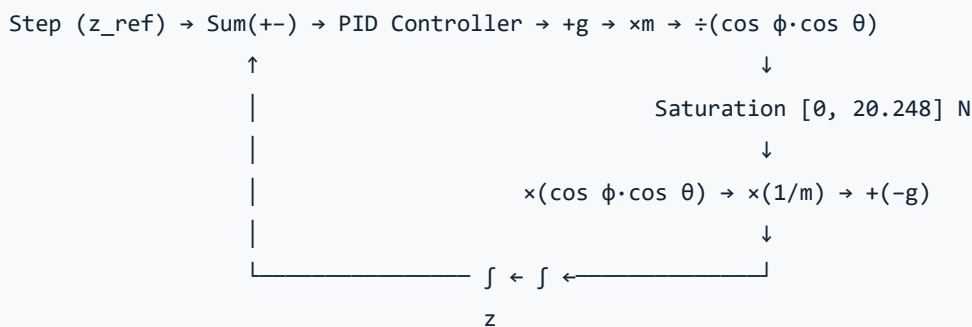
#### 07\_reference\_paper\_benchmark.png

- **Made by:** `task2b_zn_tyreus_luyben.m`
- **Shows:** upper panel — step to 2.0 m for all four designs; lower panel — the same designs under a 0.30 rad pitch step at  $t = 10$  s
- **How:** each design is simulated in the configuration its author intended — the three from Mien & Tu as plain PID (`use_fbl = false`, matching their Eq. 27), ours feedback-linearised
- **Point:** we have the lowest ITAE of all four, and  $253\times$  less sag under tilt.

## 4. The Simulink model

```
open_system('quad_altitude')
```

Structure, left to right:



| Block                      | Value                        | Meaning                                                                                             |
|----------------------------|------------------------------|-----------------------------------------------------------------------------------------------------|
| PID                        | Kp=30.1847, Ki=0, Kd=10.3994 | from <code>task2_pid.mat</code>                                                                     |
| <code>mass</code> gain     | 0.516                        | m, converting desired acceleration → force                                                          |
| <code>inv_mass</code> gain | 1.938                        | <b>1/m</b> — Simulink's Gain block multiplies, so dividing by m means multiplying by its reciprocal |
| <code>gravity</code>       | -9.81                        | so the vehicle falls with no thrust                                                                 |
| <code>g_ff</code>          | +9.81                        | gravity feed-forward, added <i>before</i> the mass gain                                             |
| <code>U1_sat</code>        | [0, 20.248] N                | the rotors' real limit                                                                              |
| <code>theta_cmd</code>     | 0.3 rad step at t = 4 s      | the tilt disturbance                                                                                |

**Solver:** fixed-step `ode3`, dt = 0.001, stop 12 s. **Parameters load automatically** via the model's `PreLoadFcn` callback ( `P = quad_params();` ).

**What to point at during a demo:** run it, open the Scope, and note that at t = 4 s the pitch steps to 17.2° and **altitude does not move** — thrust rises from 5.062 N to 5.30 N (= 5.062/cos 0.3) to compensate exactly.

**To rebuild after changing gains:** `build_simulink_model` — it reads `task2_pid.mat`, so the model can never drift out of sync with the tuning.

## 5. Reproducing the headline numbers

| Claim                                     | Command that proves it                                               |
|-------------------------------------------|----------------------------------------------------------------------|
| <code>max\ A - A_num\  = 1.636e-12</code> | <code>task1_statespace</code> or <code>verify_all</code> section [B] |
| <code>rank(ctrb) = 12</code>              | <code>task1_statespace</code> or <code>verify_all</code> section [D] |
| Final gains, Ts = 0.95 s, OS = 0.00%      | <code>task2_altitude_pid</code>                                      |
| Sag 0.000134 m at 17.2°                   | <code>verify_all</code> section [F]                                  |
| Simulink sag 0.000003 m                   | <code>verify_all</code> section [H]                                  |
| Beats all three paper designs             | <code>task2b_zn_tyreus_luyben</code>                                 |
| Ki rises with payload                     | <code>task3b_retrain_compare</code> + figure 06                      |

## 6. Troubleshooting

**"Undefined function 'pidtune'"** — Control System Toolbox not on the path. Run `ver`.

**parfor runs serially** — Parallel Computing Toolbox missing. Everything still works, just ~4× slower.

`task3_generate_data_train_ml` prints `workers: 0` when this happens.

**Task 3 takes far longer than expected** — it is 150 conditions × up to 180 optimiser evaluations × one 8-second simulation each ≈ 345 million derivative evaluations. Set `FAST = true` (already the default) or

reduce `N` at the top of the script.

**Figures come out on a dark background** — R2026a defaults to a dark theme. `export_figures` forces white; use it rather than saving from a figure window directly.

**Simulink model gains do not match the docs** — run `build_simulink_model` to resync from `task2_pid.mat`.

**`corr` undefined** — Statistics Toolbox is not installed. We use `corrcoef` from base MATLAB instead; this is deliberate.

## 7. What each MATLAB file does

### Model (Task 1)

| File                            | Role                                                                    |
|---------------------------------|-------------------------------------------------------------------------|
| <code>quad_params.m</code>      | All Table 1 parameters + derived values. <b>Single source of truth.</b> |
| <code>quad_dynamics.m</code>    | Nonlinear 6-DOF equations: 12 states in, $\dot{x}$ out                  |
| <code>rotor2U.m</code>          | Rotor speeds $\rightarrow$ U1–U4 via $\tau = r \times F$                |
| <code>task1_statespace.m</code> | Builds and verifies A, B, C, D                                          |

### Control (Task 2)

| File                                   | Role                                                      |
|----------------------------------------|-----------------------------------------------------------|
| <code>task2_altitude_pid.m</code>      | Four tuning methods; produces the final gains             |
| <code>task2b_zn_tyreus_luyben.m</code> | Benchmark against Mien & Tu (2024)                        |
| <code>sim_altitude.m</code>            | The nonlinear closed-loop simulator used by everything    |
| <code>perf_metrics.m</code>            | Tr, Ts, OS, SSE, ITAE, IAE, ISE, control effort           |
| <code>alt_cost.m</code>                | The objective Method D and all Task 3 labels minimise     |
| <code>build_simulink_model.m</code>    | Generates <code>quad_altitude.slx</code> programmatically |

### Machine learning (Tasks 3–4)

| File                                        | Role                                                                                    |
|---------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>task3_generate_data_train_ml.m</code> | 150 conditions $\rightarrow$ features + optimal-gain labels $\rightarrow$ trained model |
| <code>task3b_retrain_compare.m</code>       | Four model classes, selected on held-out data                                           |
| <code>task3c_relabel_warmstart.m</code>     | Re-labels after a cost change, warm-started                                             |
| <code>task4_test_ml_selftuning.m</code>     | Two-phase self-tuner, 5 test cases, three-way comparison                                |

## Validation and output

| File                          | Role                                                   |
|-------------------------------|--------------------------------------------------------|
| <code>smoke_test.m</code>     | 10-point check — run after any edit                    |
| <code>verify_all.m</code>     | 25 independent checks — re-derives rather than replays |
| <code>export_figures.m</code> | All figures as white-background 150 dpi PNGs           |
| <code>run_all.m</code>        | Tasks 1–4 in order                                     |

## 8. Key options in `sim_altitude.m`

This one function powers every simulation, so it is worth knowing its options:

| Option                                                                      | Default                             | Effect                                                                                                                                |
|-----------------------------------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>use_fbl</code>                                                        | <code>true</code>                   | feedback linearisation on/off. <b>Set false to reproduce the reference paper's plain-PID configuration.</b>                           |
| <code>m_ctrl</code>                                                         | <code>P.m</code>                    | the mass the <i>controller assumes</i> . Set to nominal while <code>P.m</code> holds the true mass to model an <b>unknown payload</b> |
| <code>tilt_phi</code> ,<br><code>tilt_theta</code>                          | <code>@(t)</code><br><code>0</code> | roll/pitch profiles, as function handles                                                                                              |
| <code>dist</code>                                                           | <code>@(t)</code><br><code>0</code> | vertical disturbance force in N (wind, gusts)                                                                                         |
| <code>noise_std</code>                                                      | <code>0</code>                      | altitude measurement noise                                                                                                            |
| <code>z0</code> , <code>zdot0</code> , <code>I0</code> ,<br><code>t0</code> | <code>0</code>                      | initial conditions, including integrator state for bumpless gain switching                                                            |
| <code>nsub</code>                                                           | <code>10</code>                     | RK4 substeps per control period. <b>4 is what Tasks 3 and 4 use</b> — evaluation fidelity must match training fidelity                |

Example — heavy payload the controller does not know about, with wind:

```
P = quad_params();
P.m = P.m * 1.8; P.W = P.m * P.g;           % true mass
opt = struct('m_ctrl', 0.516, ...           % controller still assumes nominal
            'dist', @(t) 0.5, ...           % 0.5 N steady wind
            'tilt_theta', @(t) 0.2*(t>1.5));
R = sim_altitude([30.1847 0 10.3994], P, 1.0, 8, opt);
M = perf_metrics(R.t, R.z, 1.0, R.U1);
fprintf('settles at %.4f m, SSE %.4f m\n', R.z(end), M.SSE);
```